

FULL STACK DEVELOPER PATH

Chapter 4 Node.js & Back-End

Not a language · an environment · JavaScript on the server



SOURCE : freeCodeCamp × Scrimba – “Become a Fullstack Developer from Scratch”

MODULES : Node API (21:19:42) · Fullstack Node (22:42:51)

COVERS : npm · http · routing · fs & path · POST · sanitising · events

PROJECTS : Wild Horizons API · From The Other Side

⚡ THE KEY IDEA

Node is not a language — it is an environment where you write ordinary JavaScript. There is no browser, so no DOM; in exchange you get **core modules** that read files, resolve paths and serve HTTP.

1 Running JavaScript in Node

★★★★★

```
# run a file
node server.js

# the REPL — type JS live
node
```

- **REPL** = Read Evaluate Print Loop.
- **undefined** just means the line returned nothing.
- **Ctrl + C** twice exits — it stops anything running in the terminal.
- The **↑ arrow** recalls your last command.

⚠ NO DOM HERE

Logging **document** in Node throws an error — there is no browser, so **no document object**. The novelty of writing JS in the terminal wears off fast; what we actually want is to **build servers**.

2 package.json

★★★★★

DEFINITION

package.json sits at the heart of every Node project — it is the **project's blueprint**. It holds metadata (name, version, author, description), manages **dependencies**, and defines a **start script**.

```
# build it with the wizard
npm init

# then start the app
npm start
```

```
{
  "name": "wild-horizons",
  "main": "server.js",
  "type": "module",
  "scripts": { "start": "node server.js" }
}
```

init is short for **initialize**. Hit **enter** to accept a default — sometimes the default is simply blank, which is fine.

⚠ ENABLE MODULAR JAVASCRIPT

Using **import** without setting this gives: to load an ES module, set type module in the package.json. Add **"type": "module"**. The old CommonJS **require** keyword is **not used in this course**.

⚡ WHY npm start MATTERS

Collaborators shouldn't have to hunt for your entry point. **npm start** is a **generic way to launch any project**, whatever the main file is called.

TIDYING THE GENERATED FILE

npm init writes a **"test"** script you aren't going to use — delete it. The **entry point** prompt is often pre-filled with **server.js** already; the **author** field is worth actually filling in.

⚠ TWO MODULE SYSTEMS — DON'T MIX THEM

Older tutorials use **const http = require("http")** — the **CommonJS** style. This course uses **ES modules** (**import** / **export**) throughout. Recognise **require** when you meet it in an old codebase, but **don't reach for it yourself**.

3 Creating a server

★★★★★

```
import http from "node:http";

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  res.statusCode = 200;
  res.end(JSON.stringify(destinations));
});

server.listen(8000, () => console.log("Server running on port 8000"));
```

THE node: PREFIX

Best practice is `"node:http"`, not `"http"`. It tells Node this is a **core module**, not one of your own files – which **speeds up resolution and prevents errors** in large projects.

► req and res

Member	What it does
<code>req.url</code>	The path requested – <code>/api/continent/asia</code>
<code>req.method</code>	<code>"GET"</code> , <code>"POST"</code> , ...
<code>res.setHeader()</code>	A method – set the content type
<code>res.statusCode</code>	A property , not a method – just assign to it
<code>res.end()</code>	Send the body and close the response

⚠ STRINGIFY BEFORE SENDING

You cannot send a JavaScript object down the wire. Always `JSON.stringify()` it, and set `Content-Type: application/json` so the client knows **how to interpret what arrives**.

► Handling the unhappy path

```
else {
  res.setHeader("Content-Type", "application/json");
  res.statusCode = 404;
  res.end(JSON.stringify({
    error: "Not Found",
    message: "The requested route does not exist"
  }));
}
```

⚡ DON'T ONLY CODE THE HAPPY PATH

It is easy to assume users always behave predictably. Decide deliberately **how much feedback to give** when someone misuses an endpoint – that is a real API design question, not an afterthought.

4 Routes & path parameters

★★★★★

`/api/continent/africa`

the API
base route

filter by...
location type

...this value
path parameter

Fig 1 — anatomy of a path-parameter route

```
if (req.url === "/api" && req.method === "GET") {
  // send everything
} else if (req.url.startsWith("/api/continent") && req.method === "GET") {
  const continent = req.url.split("/").pop();

  const filtered = destinations.filter(d =>
    d.continent.toLowerCase() === continent.toLowerCase()
  );
}
```

Technique	Why
<code>.startsWith()</code>	The route has a variable tail, so <code>===</code> won't do
<code>.split("/").pop()</code>	Turn the URL into an array, take the <i>last</i> segment
<code>.toLowerCase()</code> both sides	Guards against inconsistent casing in the data
Check <code>req.method</code> too	The same URL may mean different things for GET vs POST

► Refactoring — DRY

```
// utils/sendJsonResponse.js
export const sendJsonResponse =
  (res, statusCode, payload) => {
    res.setHeader("Content-Type",
      "application/json");
    res.statusCode = statusCode;
    res.end(JSON.stringify(payload));
  };
```

⚡ CLEAR BEATS CLEVER

When refactoring, aim for *clean, easy-to-read code* — that matters more than clever, complex but shorter code. You should be able to *see your routes clearly* at a glance.

NAMING

The third argument is called the *payload* — a widely used term. `body` or `data` would be fine too.

Long, descriptive file names like

`getDataByPathParams` beat short cryptic ones.

Starting with a *verb* signals that it returns something.

5 Reading files - three ways

★★★★★

DEFINITION

The **file system module** (`fs`) provides methods for interacting with your operating system's file system – `read`, `write`, update, delete and rename files. This course only needs **reading and creating**.

There are **multiple ways** to call the same fs methods, and that causes confusion. Two of them you will meet in existing code but should not reach for first:

Pattern	Import	Verdict
Sync	<code>node:fs</code> → <code>readFileSync</code>	<i>Blocking.</i> Harmless in a tiny app; performance problems in a big one
Callback	<code>node:fs</code> → <code>readFile(p, cb)</code>	Async, but leads to callback hell – hard to maintain and debug
Promises ✓	<code>node:fs/promises</code>	Modern. Easy to read and write with async / await

```
// X blocking
const content = fs.readFileSync(filePath);
```

```
// X callback hell awaits
fs.readFile(filePath, (err, content) => {
  if (err) { console.log(err); return; }
  res.end(content);
});
```

```
// ✓ the one to use
import fs from "node:fs/promises";
```

```
const server = http.createServer(
  async (req, res) => {
    const content = await fs.readFile(filePath);
    res.end(content);
  });
```

REMEMBER

Adding **`import { promises } from 'node:fs'`** to the import is a technique **Node provides to wrap a promise around the fs module**. Many Node modules ship both sync and async versions of a method – **prefer async** for the non-blocking benefits.

► The encoding trap – send a buffer

- Encoding is an **optional** 2nd argument: `readFile(path, "utf8")`.
- With **`utf8`** you get a **string** – the HTML in string form.
- Without it you get a **Buffer**.

⚠ LEAVE THE ENCODING OFF

Your serve function isn't only for HTML – it serves **images** too, which don't need UTF-8. Applying UTF-8 across the board could **corrupt some files**.

Browsers interpret buffers correctly **using the Content-Type header**. So send the buffer: it makes the server more flexible. When in any doubt about encoding, omit it and let Node and the browser handle it.

BUFFER

A data type in Node – a **low-level representation of raw bytes**. You can't do much with it directly; it needs encoding to be readable.

6 OS-agnostic paths

★★★★

⚠ WHY HARD-CODED PATHS BREAK

Windows separates folders with `\`; Mac and Linux use `/`. A hard-coded path also breaks when a **collaborator** clones the repo to a different location, and when you **deploy** somewhere new. You need an **OS-agnostic** solution.

```
import path from "node:path";
```

```
const __dirname = import.meta.dirname;           // Node 20+
const publicDirectory = path.join(__dirname, "public");
const filePath = path.join(publicDirectory, "index.html");
```

`import.meta` is an object available **only in modular JavaScript** – which is exactly why we set `"type": "module"`. Old CommonJS gave you a global `__dirname`, so the convention is to **mimic that name**.

OLDER CODEBASES

Before Node 20 there was no `import.meta.dirname`. You will still meet this two-step workaround – worth **recognising**, not memorising:

```
import { fileURLToPath } from "node:url";
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

► Three kinds of location

Term	Meaning
Current working directory	The folder you were in when you ran <code>node server.js</code> . Read it with <code>process.cwd()</code> . It can differ in complex deployments
Absolute path	The full location. Independent of the cwd – safe as long as the file structure doesn't change
Relative path	Relative to the file it appears in – <code>.</code> , <code>..</code>

⚠ THE SUBTLE ONE

A relative path built **with the path module starts from the current working directory**, **not** from the file it is written in. So relative paths **are** affected by cwd changes, while `import.meta.dirname` is not. That is precisely why we build from `__dirname`.

⚡ WHY path.join AND NOT STRING CONCATENATION

Gluing strings together with `+ "/" +` hard-codes the **forward slash** – the exact thing that breaks on Windows. `path.join()` inserts **whatever separator the host OS uses** and tidies up stray or doubled slashes on the way.

7 One route for every asset ★★★★★

You could write a route per file – but there could be *thousands* of assets. Instead, derive the file path from `req.url`:

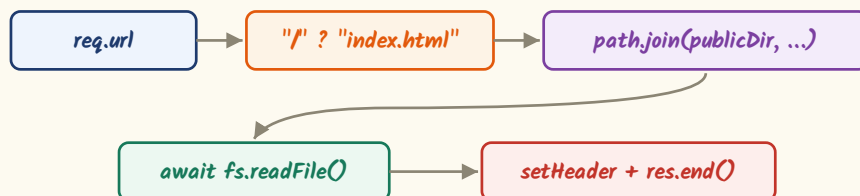


Fig 2 – the static-file pipeline

⚠ THE HOMEPAGE EXCEPTION

For the homepage the browser sends *just /* – that is not a filename. Handle it with a ternary:

```
const pathToResource = req.url === "/" ? "index.html" : req.url;
```

► Content type – the trickiest bit

```
// utils/getContentType.js
export const getContentType = (ext) => {
  const types = {
    ".html": "text/html",
    ".css": "text/css",
    ".js": "text/javascript",
    ".png": "image/png",
    ".jpg": "image/jpeg"
  };
  return types[ext.toLowerCase()] || "text/html";
};
```

⚡ WHY text/html IS THE DEFAULT

The `// "text/html"` fallback isn't laziness – a browser *handles an unknown file type best as HTML*.

⚠ CSS FAILS SILENTLY

Get it wrong for an *image* and the browser usually forgives you – it still renders. Get it wrong for *CSS* and the stylesheet *silently does nothing*. No error, just an unstyled page.

Get the extension with

`path.extname(pathToResource)`. A `switch` would work equally well. A production map would be *far longer*.

```
// utils/serveStatic.js – the whole thing, assembled
export const serveStatic = async (req, res, publicDirectory) => {
  const pathToResource = req.url === "/" ? "index.html" : req.url;
  const filePath = path.join(publicDirectory, pathToResource);
  const contentType = getContentType(path.extname(pathToResource));

  const content = await fs.readFile(filePath); // no encoding → Buffer

  res.setHeader("Content-Type", contentType);
  res.statusCode = 200;
  res.end(content);
};
```


8 Receiving data from the client

★★★★★

```
else if (req.method === "POST" && req.url === "/api") {
  handlePost(req, res);
}
```

Note we now need **req** as well as **res** – for a GET we only ever **sent** data; now we must **read** it.

THE BODY ARRIVES IN CHUNKS

The client's operating system and TCP **break the request body into chunks**. A small body may arrive as one; a larger one as many. You don't know **how many** nor how big – so the process is **inherently asynchronous**.

```
// utils/parseJsonBody.js
export const parseJsonBody = async (req) => {
  let body = "";

  for await (const chunk of req) { // req is an async iterable
    body += chunk;
  }

  try {
    return JSON.parse(body);
  } catch {
    console.log("Invalid JSON");
    return; // ← stop everything
  }
};
```

⚠ TWO THINGS PEOPLE GET WRONG

1. A plain **for...of** is **not enough** – you must use **for await...of**, because each chunk arrives asynchronously.
2. Parsing must sit in **try/catch**, and the catch must **return early** so no further code runs for that route.

► The full POST pipeline

Step	Utility
Collect the incoming data · parse it	<code>parseJsonBody</code>
Sanitise it – see page 9	<code>sanitizeData</code>
Get existing data · add the item · write back	<code>addNewSighting</code>
Reply to the client	<code>sendResponse</code>

CORS – needed the moment you deploy

Two response headers tell the browser that any domain, port or protocol may call your API. Irrelevant on one origin, **required once the front end lives elsewhere**.

9 Cross-site scripting

★★★★★

DEFINITION – XSS & SANITISATION

A **cross-site scripting** attack (XSS) is a security vulnerability that allows an attacker to **inject malicious scripts into web pages**. Type JavaScript wrapped in HTML into a form, have it rendered back, and the attack has landed.

Sanitisation = removing **anything suspicious from incoming input** – here, stripping tags out of user text.

► The dependency: sanitize-html

```
# install it
npm install sanitize-html

// the function is camelCased, the package name is not
import sanitizeHtml from "sanitize-html";
```

With no options it is already smart: `<h1>` `<strong>` `<p>` survive with their text, while `<style>` and `<script>` are removed along with their contents.

TAKING CONTROL

Pass an options object to decide exactly what lives:

```
sanitizeHtml(value, {
  allowedTags: ["b"],      // empty array = strip every tag
  allowedAttributes: {}    // empty object = allow none
})
```

This lets you **disallow HTML without penalising non-malicious users** – someone may have used a `<b>` tag simply to make part of their story stand out.

⚠ IT ONLY TAKES STRINGS

Pass an **object** and you get nothing back. Real submissions are objects, so iterate the keys and sanitise values one by one.

```
// utils/sanitizeInput.js
export function sanitizeInput(data) {
  const sanitizedData = {};
  for (const [key, value] of Object.entries(data)) {
    if (typeof value === "string") {
      sanitizedData[key] = sanitizeHtml(value, {
        allowedTags: ["b"], allowedAttributes: {}
      });
    } else {
      sanitizedData[key] = value; // numbers, timestamps... pass through
    }
  }
  return sanitizedData; // ← easy to forget
}
```

⚡ WHERE IT SLOTS IN

Not in `server.js` – **architecture matters**. In the route handler, right after parsing:

`const sanitizedBody = sanitizeInput(parsedBody)`. That object is what gets **saved** and what appears in the **response**.

10 Event-driven architecture

★★★★

DEFINITION

Node is said to have **event-driven architecture**: a programming style where **the flow of a program is determined by events**. Instead of a strict sequence of instructions, code is designed to **react to events as they occur**.

An **event** is a signal that something has happened – a user clicks a button, a file finishes downloading, a network request is received.

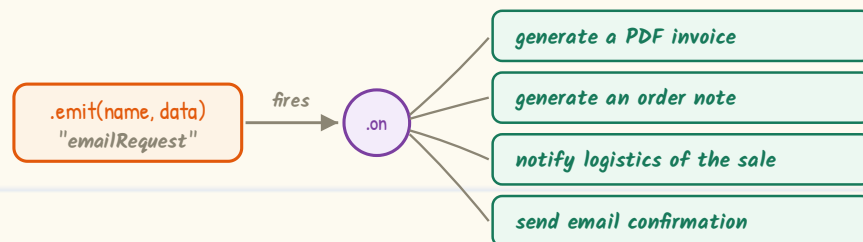


Fig 3 – one emitted event, many listener functions

```
import { EventEmitter } from "node:events";

const emailRequestEmitter = new EventEmitter();

// 1 – define a listener function
function generateEmail(customer) {
  console.log(`Email generated for ${customer.email}`);
}

// 2 – register it: "on hearing this event, do this"
emailRequestEmitter.on("emailRequest", generateEmail);

// 3 – emit it from anywhere in the app
emailRequestEmitter.emit("emailRequest", customerDetails);
```

⚠ THE TWO CRUCIAL METHODS

.emit(eventName, ...params) – you may pass **multiple** parameters, or **none** if the listener needs nothing.

.on(eventName, listener). With everything in one file this is contrived – basic function calls would do. The payoff comes **when your code is modular**: emit in one part of the app, listen in another. Easy to scale and maintain – and you can have **multiple listeners** on one event.

⚡ PROVE IT TO YOURSELF

Wrap the **.emit()** call in a **setTimeout** of two seconds and run the file. Nothing happens, **then** every listener fires at once – visible proof that **.on()** is **reacting to the event when it happens**, not simply running in source order. Extra listeners can be added inline as arrow functions.

11 Pushing data to the client

★★★★

DEFINITION – EventSource

A browser interface used to receive **push notifications** from a server via **server-sent events**. It allows a **persistent, one-way connection** where the server sends updates to the client.

► Server side

```
res.setHeader("Content-Type", "text/event-stream");
res.setHeader("Cache-Control", "no-cache");
res.setHeader("Connection", "keep-alive");

setInterval(() => {
  res.write(data: ${JSON.stringify({
    event: "newsUpdate",
    story: stories[randomIndex]
  })}\n\n);
}, 30000);
```

⚠ TWO THINGS THAT BREAK IT

1. The trailing `\n\n` is **required by the server-events protocol** – it signals the end of a complete message block. **Miss it out and this will not work.**
2. Use `res.write()`, **never** `res.end()` – **end** finishes the response, and we want to **keep the connection open**.

► Client side

```
const eventSource = new EventSource("/api/news");

eventSource.onmessage = (event) => { // NOT camelCased
  const data = JSON.parse(event.data);
  newsDisplay.textContent = data.story.headline;
};

eventSource.onerror = () => console.log("EventSource failed");
```

NAME YOUR EVENTS

The `event` property in the payload is **optional** – but with lots of streams in one codebase, a descriptive name saves confusion.

⚠ RELATIVE PATHS

Point `EventSource` at a **relative** endpoint path, not an absolute URL.

Why it matters: you are **sending data from the server down a live connection whenever you want** – no polling, no request from the client. That is the whole point of server-sent events.

12 Project 1 - Wild Horizons API ★★★★★

A travel-destinations API built with nothing but Node's core modules. What it covered:

- The **HTTP** core module
- Creating a server
- Status codes & headers
- Requests and responses
- Filtering data
- Query params & path params

STRETCH GOALS

Gentler **error handling**; add **POST** support (skip authentication for now); keyword **search** on the description field.

13 Project 2 - From The Other Side ★★★★★

A paranormal-sightings site: a vanilla-Node backend with a deliberately simple static front end (HTML, CSS, JS and images).

⚡ WHY DELIBERATELY SIMPLE

The front end is basic on purpose — you are pretty unlikely to be building a full stack project with a vanilla Node backend in the real world. The point is to understand what frameworks do for you before you let them do it.

Task	What it needed
Serve the assets	<code>fs</code> · <code>path</code> · content-type mapping
Provide data via an API	Routing on <code>req.url</code> and <code>req.method</code>
Handle a POST	Chunked body · parse · sanitise · write back to JSON

PROJECT RECAP — the five skills

1. The **file system module**, to read and write the JSON file. 2. The **path module**, for paths that work across operating systems. 3. Serving **static assets** — HTML, CSS, JS and images. 4. Working with a **dependency**, brought in to sanitise input and prevent XSS. 5. **Events** and server-sent events.

⚠ A LOOSE END WORTH FIXING

If a client requests `lapitxyz`, that URL **starts with** `lapi`, so the catch-all `else` never runs — but the exact-match `req.url === "lapi"` branch does not run either. Requests fall **between the two routes**. Tightening this up is the first stretch goal.

⚡ WHAT THIS CHAPTER WAS REALLY TEACHING

Two projects, one lesson: a server is just a function that receives a request and writes a response. Everything else — routing, static files, parsing bodies, content types — is detail you now implement by hand rather than inherit from a framework.

14



QUICK REVISION

Thing	The one-line answer
What is Node?	Not a language – an environment for running JavaScript. No browser, no DOM
<code>package.json</code>	The project's blueprint: metadata, dependencies, start script
<code>"type": "module"</code>	Enables <code>import</code> . Without it you get the ES-module error
<code>node:</code> prefix	Marks a core module – faster resolution, fewer errors
<code>res.statusCode</code>	A property. <code>res.setHeader()</code> and <code>res.end()</code> are methods
Path params	<code>.startsWith()</code> to match, <code>.split("/").pop()</code> to extract
fs, the right way	<code>import fs from "node:fs/promises" + await</code>
Encoding	Omit it. Send a Buffer; the Content-Type header tells the browser what to do
<code>import.meta.dirname</code>	Directory of the current module. Only exists in modular JS
<code>process.cwd()</code>	Where you ran the command – not where the file lives
Homepage URL	<code>req.url === "/" ? "index.html" : req.url</code>
Wrong content type	Images still render; CSS fails silently
Reading a POST body	<code>for await (const chunk of req)</code> – it arrives in chunks
<code>JSON.parse</code>	Always in <code>try/catch</code> , and <code>return</code> early on failure
XSS defence	<code>sanitize-html</code> , iterating object keys – it only accepts strings
EventEmitter	<code>.emit(name, data)</code> fires · <code>.on(name, fn)</code> listens
SSE must-haves	<code>text/event-stream</code> · <code>res.write</code> not <code>end</code> · trailing <code>\n\n</code>

⚡ THE FIVE MOST FORGETTABLE DETAILS

1. `for...of` without `await` silently fails on a request body.
2. Missing `\n\n` silently kills server-sent events.
3. UTF-8 encoding corrupts images.
4. Relative paths built with the path module start from the `cwd`, not the file.
5. `sanitizeHtml` returns nothing when handed an object.

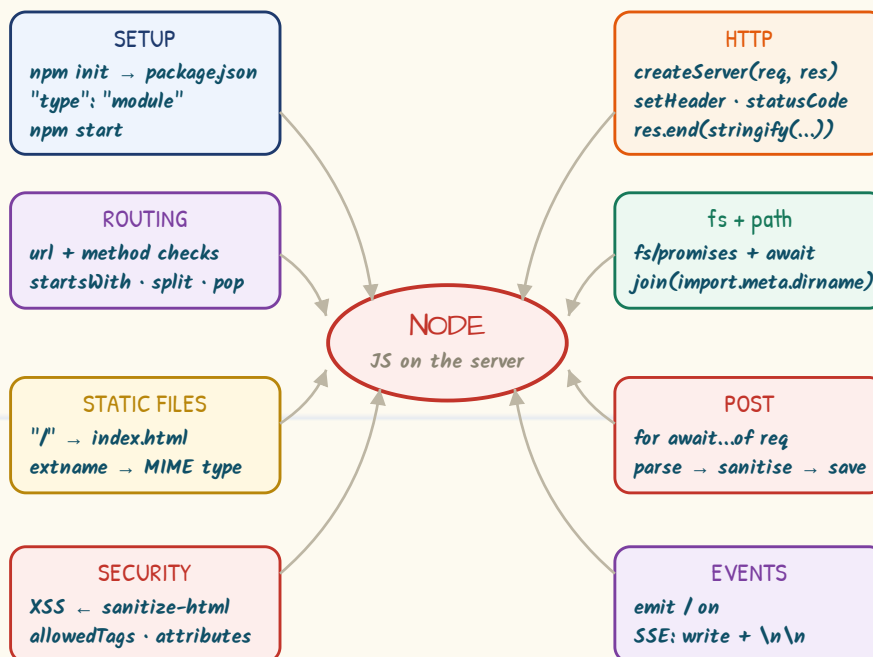
METHOD vs PROPERTY – a classic slip

`res.statusCode = 200` uses an equals sign; `res.setHeader("Content-Type", ...)` and `res.end(...)` use parentheses. Writing `res.statusCode(200)` is the mistake to watch for.

STATUS CODES USED IN THIS CHAPTER

200 – OK, here is your data. 201 – Created, after a successful POST. 404 – Not Found, the requested route does not exist. 500 – the server itself broke. If you come from front-end dev, it is quite nice to be the one giving out the 404 rather than receiving it.

15 MIND MAP



⚡ THE THROUGH-LINE

Every one of these is plain JavaScript plus a core module. No framework has appeared yet – which is exactly the point. When you meet Express later, you will recognise every single thing it is doing **for** you, because you have already done it **by hand**.

Core module	Imported as	Used for
<code>http</code>	<code>"node:http"</code>	<code>createServer · listen · req / res</code>
<code>fs (promises)</code>	<code>"node:fs/promises"</code>	<code>readFile · writeFile</code> , with <code>await</code>
<code>path</code>	<code>"node:path"</code>	<code>join · extname · dirname</code>
<code>events</code>	<code>"node:events"</code>	<code>EventEmitter</code> → <code>emit / on</code>
<code>url</code>	<code>"node:url"</code>	<code>fileURLToPath</code> – only pre-Node-20
<code>sanitize-html</code>	npm dependency	The one non-core package in the project

Next up → **databases** and **SQL**, so the data outlives a JSON file.